

Gerenciamento de módulos

Linguagens de programação populares tal como o Python são mantidas por comunidades de pessoas e organizações interessadas em seu crescimento e disseminação nas diferentes soluções da tecnologia da informação. Entender o funcionamento da comunidade é importante para compreender como os módulos são desenvolvidos e disseminados por ela. Este capítulo, em essencial, trata das informações pertinentes ao gerenciamento dos inúmeros módulos existente na comunidade de desenvolvedores Python.

O primeiro subcapítulo trata de como é formada a comunidade de desenvolvedores do Python, justificando sua grande disseminação nas diversas aplicações voltadas à tecnologia da informação. Depois, será apresentado como utilizar o gerenciador de módulos pip, imprescindível para a gestão de ferramentas em Python, assim como entender a importância e o funcionamento dos ambientes virtuais. Ao final, entenderemos como criamos nossos próprios módulos para serem reaproveitados nas diferentes aplicações.

1 A comunidade de desenvolvimento Python

Desde que Guido van Rossum criou a linguagem Python, em 1989, ela tem evoluído significativamente, não se limitando ao aspecto técnico. Python cresceu como reflexo de uma comunidade colaborativa que ajudou a moldar a cultura moderna do desenvolvimento de software. Desde o início, a proposta foi clara: criar uma linguagem *simples, legível e acessível*. E foi justamente essa combinação que conquistou tanto quem está começando quanto programadores experientes.

Com o tempo, a comunidade de desenvolvedores Python se firmou como uma das mais inclusivas da programação. Eventos como a PyCon acontecem em vários países, e iniciativas como a Python Software Foundation (PSF) são exemplos de como a linguagem se apoia em valores como diversidade, educação e inovação (Pycon, 2025; Python, 2025). A filosofia expressa no Zen of Python, especialmente o princípio “There should be one – and preferably only one – obvious way to do it” (“Deve haver uma – e preferencialmente apenas uma – maneira óbvia de fazer isso”), reforça uma cultura que preza pela clareza, por boas práticas e pelo trabalho em conjunto.

Um repositório importante para a comunidade de desenvolvedores é o Python Package Index (PyPI), o repositório oficial de pacotes de software para a linguagem de programação Python, desempenhando um papel fundamental no ecossistema da linguagem ao facilitar o compartilhamento e a reutilização de código. Por meio dele, desenvolvedores do mundo todo podem publicar bibliotecas, frameworks e utilitários que resolvem desde tarefas simples até problemas altamente complexos. Ao centralizar esses recursos em um único local, o PyPI promove a colaboração e o avanço da comunidade Python e acelera o desenvolvimento de projetos ao oferecer soluções prontas e bem documentadas. Dessa forma, o PyPI se consolida como uma ferramenta indispensável para quem deseja programar de forma eficiente, escalável e moderna (PyPI, 2015).

No Brasil, o uso de Python ganhou força a partir dos anos 2000, impulsionado por iniciativas da comunidade e por seu uso crescente em universidades e projetos de código aberto. A criação da Python Brasil (conferência nacional que reúne entusiastas, estudantes e profissionais) foi um marco importante para consolidar o ecossistema no país (Python Brasil, 2015). Além disso, grupos locais como os PUGs (Python User Groups) espalhados por diversas cidades ajudaram a democratizar o conhecimento, promovendo encontros, workshops e maratonas de programação. Python se popularizou entre cientistas de dados, desenvolvedores web e pesquisadores, tornando-se uma linguagem estratégica em setores como educação, governo, fintechs e tecnologia ambiental. Esse movimento comunitário, unido à versatilidade da linguagem, fez com que o Brasil se destacasse como uma das comunidades Python mais ativas do mundo.



PARA SABER MAIS

Python é uma linguagem conhecida por sua simplicidade e versatilidade, muito graças ao seu vasto ecossistema de frameworks, que ajudam a acelerar o desenvolvimento e aplicar boas práticas desde o início. Entre os mais populares está o Django, um framework completo para aplicações web, que já vem com autenticação, painel administrativo, ORM (object-relational mapping ou mapeamento objeto-relacional) e muito mais. O Flask, por sua vez, é mais leve e flexível, ideal para projetos menores ou quando se busca maior controle sobre os componentes utilizados. O FastAPI vem ganhando destaque como uma opção moderna e assíncrona para a construção de APIs rápidas e bem documentadas. Para quem trabalha com dados, o Pandas e o NumPy são ferramentas essenciais para manipulação e cálculos numéricos. Já no universo da inteligência artificial, frameworks como TensorFlow e PyTorch lideram o cenário, sendo o primeiro mais comum em ambientes de produção, e o segundo preferido em pesquisas por sua flexibilidade. E para quem deseja automatizar a extração de dados da web, o Scrapy é uma excelente escolha. Cada framework atende a um tipo de necessidade, e conhecer essas ferramentas é o primeiro passo para ampliar as possibilidades com Python e desenvolver projetos mais eficientes e profissionais.

2 O gerenciamento de módulos com pip

O ecossistema do Python é amplamente conhecido por sua riqueza de bibliotecas e pacotes que estendem as funcionalidades da linguagem para diferentes finalidades: desenvolvimento web, análise de dados, inteligência artificial, automação, entre muitas outras. Para lidar com essa diversidade, o Python utiliza o pip (acrônimo para `pip installs packages`) como seu principal gerenciador de pacotes. O pip é uma ferramenta que permite instalar, atualizar e remover pacotes Python com facilidade. Ele é incluído por padrão nas versões mais recentes do Python (a partir da 3.4), mas pode ser atualizado para a versão mais recente com o seguinte comando:

```
python -m pip install --upgrade pip
```

Para verificar a versão instalada, basta utilizar o comando pip com a opção “version”, como segue:

```
pip --version
```

A instalação de pacotes com o pip é realizada utilizando o comando `pip install`, adicionando também o nome do pacote que precisa ser instalado. Por exemplo, para instalar a biblioteca `requests` na versão mais recente, deve ser utilizado o comando:

```
pip install requests
```

Também é possível especificar a versão desejada para o pacote, através do comando:

```
pip install requests==2.31.0
```

Existem outras opções para a instalação de pacotes a partir de pastas locais, URLs ou arquivos “.whl” (wheel), conforme os exemplos a seguir, respectivamente.

Exemplo 1

```
pip install ./meu_pacote/  
pip install https://exemplo.com/arquivo.tar.gz  
pip install pacote_exemplo-1.0.0-py3-none-any.whl
```

Para verificar os pacotes instalados no ambiente atual, deve ser utilizado o comando pip com a opção list:

```
pip list
```

É possível obter detalhes de um pacote específico, como versão, dependências e localização, pela opção show:

```
pip show nome-do-pacote
```

Também é possível verificar pacotes desatualizados com a opção outdated:

```
pip list --outdated
```

Pacotes podem ser atualizados com a opção upgrade, adicionando ao final o nome do pacote, como apresentado a seguir.

```
pip install --upgrade nome-do-pacote
```

Para remover um pacote, pode ser usado o comando uninstall, adicionando ao final o nome do pacote, como mostrado na sequência

```
pip uninstall nome-do-pacote
```

Outro aspecto importante é a geração de um arquivo com as dependências de um ambiente para garantir que a execução de um aplicativo ou serviço seja realizada nas mesmas condições. Para isso, é comum utilizar um arquivo chamado requirements.txt, o qual pode ser gerado pela opção freeze, apontando a saída do arquivo para ser gravado em requirements.txt, como segue.

```
pip freeze > requirements.txt
```

Para instalar os pacotes de maneira a reproduzir o ambiente com todas as dependências, basta usar o comando `install` com a opção `-r` e o nome do arquivo com as dependências (na maioria dos casos, o `requirements.txt`).

```
pip install -r requirements.txt
```

Dominar o uso do `pip` é essencial para qualquer profissional que desenvolva com Python. Além de facilitar a instalação e manutenção de bibliotecas, ele promove organização, reprodutibilidade e escalabilidade nos projetos.

3 Ambiente virtual

Uma das boas práticas mais importantes no desenvolvimento em Python é a utilização de ambientes virtuais, que isolam as dependências de cada projeto, evitando conflitos entre versões de pacotes.

Para criar um ambiente virtual, basta utilizar o módulo `venv` do Python, indicando nome do ambiente. Serão criados uma pasta referente ao nome do ambiente e os arquivos internos para instanciar o ambiente em seu sistema operacional, como segue.

```
python -m venv nome_do_ambiente
```

Para ativar o ambiente virtual, o comando vai depender do sistema operacional que está sendo utilizado. Caso seja o Windows, o script de ativação pode ser ativado utilizando o terminal CMD ou PowerShell, como segue.

No CMD:

```
nome_do_ambiente\Scripts\activate
```

No PowerShell:

```
nome_do_ambiente\Scripts\Activate.ps1
```

Para ativar no Linux ou no MacOS:

```
source nome_do_ambiente/bin/activate
```



IMPORTANTE

Fique atento a permissões específicas para execução do ambiente virtual que o sistema operacional pode solicitar!

É importante ressaltar que ambientes virtuais podem ser utilizados para garantir que os conjuntos de dependências de uma aplicação estejam devidamente apartados do host principal, garantindo a instalação de pacotes fora de contexto que podem aumentar o tamanho da aplicação e até mesmo gerar conflitos de pacotes no momento da execução. O uso de ambientes virtuais, associados a arquivos de requisitos (por exemplo, o requirements.txt) são práticas recomendadas que tornam o trabalho mais limpo, seguro e profissional.

O ambiente virtual pode a qualquer momento ser desativado utilizando o comando deactivate, bastando somente executar o comando, como segue:

```
deactivate
```

4 Criando seus próprios módulos

Além de consumir pacotes criados por outros desenvolvedores, o Python também permite que você crie seus próprios módulos e os torne instaláveis via pip. Essa prática é útil tanto para organizar seus projetos quanto para compartilhar bibliotecas com outras pessoas ou times.

Um pacote Python precisa seguir uma estrutura mínima para ser reconhecido corretamente. Por exemplo:

```
meu_pacote/  
  
    meu_pacote/          Diretório com o código fonte  
        __init__.py  
        modulo.py  
  
    README.md  
    setup.py  
    pyproject.toml  
    LICENSE
```

Cada arquivo dentro da estrutura de pastas possui diferente papéis, como segue:

- **__init__.py**: indica que o diretório é um pacote Python.
- **setup.py**: define metadados e instruções de instalação.
- **pyproject.toml**: especifica as dependências de build (recomendado para novas versões do Python).
- **README.md**: documentação básica.
- **LICENSE**: tipo de licença (MIT, GPL etc.).

Para versões anteriores do Python, é utilizando o arquivo setup.py para configurar a instalação, definir metadados e instalar as dependências, conforme a seguir.

```
from setuptools import setup, find_packages  
  
setup(  
    name='meu_pacote',  
    version='0.1.0',  
    description='Um pacote de exemplo para demonstração com  
pip',  
    author='Seu Nome',
```

```
author_email='seu@email.com',
packages=find_packages(),
install_requires=[], # dependências, se houver
classifiers=[
    'Programming Language :: Python :: 3',
    'License :: OSI Approved :: MIT License',
],
)
```

Explicando a composição do arquivo setup.py:

- **setuptools:** é uma biblioteca que substitui o antigo distutils e oferece ferramentas mais modernas e flexíveis para empacotar projetos Python.
- **find_packages():** detecta automaticamente todos os pacotes e subpacotes dentro de seu projeto (ou seja, diretórios com `__init__.py`). Isso evita a necessidade de listá-los manualmente.

Na função `setup()`, existem os seguintes parâmetros:

- **name:** nome do pacote como ele será reconhecido no pip e no PyPI. Evite espaços e caracteres especiais.
- **version:** versão atual de seu pacote. É uma boa prática seguir o versionamento semântico (ex: MAJOR.MINOR.PATCH).
- **description:** uma descrição curta e clara do que seu pacote faz.
- **author / author_email:** informações sobre quem criou o pacote. Isso é exibido no PyPI e ajuda a identificar o responsável.
- **packages:** lista dos diretórios que contêm código-fonte. `find_packages()` busca automaticamente por eles. Você também pode passar uma lista manual, como `['meu_pacote']`.
- **install_requires:** lista de pacotes que seu projeto depende para funcionar corretamente. Eles serão instalados automaticamente com seu pacote.

- **classifiers:** metadados padronizados utilizados pelo PyPI para classificar e descrever seu pacote. Eles ajudam na busca por pacotes com características específicas.



IMPORTANTE

Muitos projetos usam o `pyproject.toml` como padrão moderno. O `setup.py` ainda é aceito, mas pode ser substituído gradualmente.

Nos casos em que se opte pelo uso do arquivo `pyproject.toml`, segue um exemplo de preenchimento:

Exemplo 2

```
[build-system]
requires = ["setuptools>=61.0", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "meu_pacote"
version = "0.1.0"
description = "Um pacote de exemplo para demonstração com pip"
authors = [
    { name = "Seu Nome", email = "seu@email.com" }
]
license = { text = "MIT" }
readme = "README.md"
requires-python = ">=3.8"
dependencies = [
    "requests>=2.25",
    "pandas>=1.0,<2.0"
]

[project.urls]

"Homepage" = "https://github.com/seunome/meu_pacote"
"Documentação" = "https://meu-pacote.readthedocs.io"
```

Explicando a composição do arquivo `pyproject.toml`:

- `[build-system]`
- Define o backend de empacotamento e suas dependências mínimas.
- `setuptools` e `wheel` são os mais comuns.
- O `setuptools>=61.0` é necessário para suportar o novo formato via `pyproject.toml`.
- `[project]`
- Metadados do pacote: nome, versão, descrição, autor, licença etc.
- `requires-python`: especifica a versão mínima do Python.
- `dependencies`: lista de pacotes necessária para o funcionamento do projeto.
- `[project.urls]` (opcional)
- URLs úteis: site do projeto, documentação, repositório GitHub etc.

Para empacotar a solução e instalar, o próximo passo é gerar o arquivo de distribuição, através do comando:

```
python -m build
```

Para instalá-lo localmente, deve ser usado o comando `pip` buscando arquivos para a instalação local com o comando:

```
pip install .
```

O pacote também pode ser publicado no PyPI através do comando `twine`, de maneira que qualquer pessoa possa instalar seu pacote remotamente:

```
twine upload dist/*
```



PARA SABER MAIS

Publicar pacotes no PyPI (Python Package Index) é o passo final para compartilhar seu código Python com a comunidade de forma padronizada e acessível via pip install. Para isso, é necessário ter uma conta no PyPI, criar um arquivo `pyproject.toml` ou `setup.py` com os metadados do pacote, gerar os arquivos de distribuição usando a ferramenta build e, por fim, fazer o upload com o twine. Com esses passos, qualquer pessoa poderá instalar seu pacote diretamente do PyPI, o que facilita a reutilização de código, a colaboração entre desenvolvedores e a disseminação de soluções úteis de forma segura e organizada.

Considerações finais

O gerenciamento de módulos em Python representa uma das competências fundamentais para quem deseja desenvolver aplicações organizadas, eficientes e de fácil manutenção. Ao compreender o funcionamento da comunidade Python, utilizar o pip de forma adequada e adotar ambientes virtuais para isolar dependências, o desenvolvedor assegura maior controle e confiabilidade no ciclo de vida de seus projetos. Além disso, a criação e a publicação de módulos próprios ampliam as possibilidades de reutilização e colaboração, contribuindo para o fortalecimento do ecossistema Python. Dominar essas práticas não apenas facilita o desenvolvimento, como promove a profissionalização do trabalho com software livre e de código aberto.

Referências

PYCON. **[Página inicial]**. [S. l.]: PyCon, 2025. Disponível em: <https://pycon.org/>. Acesso em: 8 abr. 2025.

PYPI. **Python Software Foundation**. [S. l.]: PyPI, 2015. Disponível em: <https://www.python.org/psf-landing/>. Acesso em: 8 abr. 2025.

PYTHON BRASIL. **[Página inicial]**. [S. l.]: Python Brasil, 2015. Disponível em: <https://python.org.br/>. Acesso em: 8 abr. 2025.

PYTHON. **Python Software Foundation**. [S. l.]: Python, 2025. Disponível em: <https://www.python.org/psf-landing/>. Acesso em: 8 abr. 2025.

Sobre o autor

Gustavo Moreira Calixto possui graduação em tecnologia em informática pela Universidade Estadual de Campinas, mestrado em engenharia elétrica pela Universidade Estadual de Campinas e doutorado em ciências pela Escola Politécnica da Universidade de São Paulo, com ênfase em sistemas eletrônicos. Na Universidade de São Paulo, atuou como pesquisador do Centro Interdisciplinar de Tecnologias Interativas (CITI) na área de sistemas embarcados e sistemas multimídia, sendo responsável pela gerência do projeto Global ITV, uma colaboração entre instituições brasileiras e europeias apoiada pelo CNPq e FP7. Atuou como pesquisador visitante na The University of Texas at Austin (EUA), na Moody College of Communication, Department of Electrical and Computer Engineering. Atualmente, é docente no Centro Universitário do Senac-SP, coordenando cursos de graduação da área de TI na modalidade EaD. É professor visitante dos cursos de pós-graduação em engenharia de dados e DevOps da Universidade Presbiteriana Mackenzie. Também foi professor doutor na Fiap nos cursos de graduação e professor assistente no Insper no curso de engenharia da computação. Foi professor no Centro Paula Souza (Fatec), ministrando para os cursos de análise de sistemas e informática para negócios. Na Universidade de Guarulhos, atuou como professor doutor no curso de ciência da computação e tecnologia em gestão de TI. Tem experiência na área de tecnologia da informação e telecomunicações, voltadas a sistemas embarcados. O profissional atuou em projetos de desenvolvimento de soluções de inovação e soluções comerciais em grupos de pesquisas e consultorias da área de informática. Foi professor e coordenador adjunto dos cursos de informática da Universidade Nove de Julho e professor mestre na Faculdade Impacta de Tecnologia. Desde 2005, realiza atividades voluntárias junto ao IEEE. Desde julho de 2017 é gerente de tecnologia da Prodam/SP.

Currículo Lattes: <http://lattes.cnpq.br/6998242767220750>

